SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	Yoga Madha A	**Reg. No.:**	192425058
Section / Batch:	B	**Date:**	16/10/2026

Code of Conduct

I Yoga Madha A/192425058 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Yoga Madha A

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Annexure A – Application Based Questions

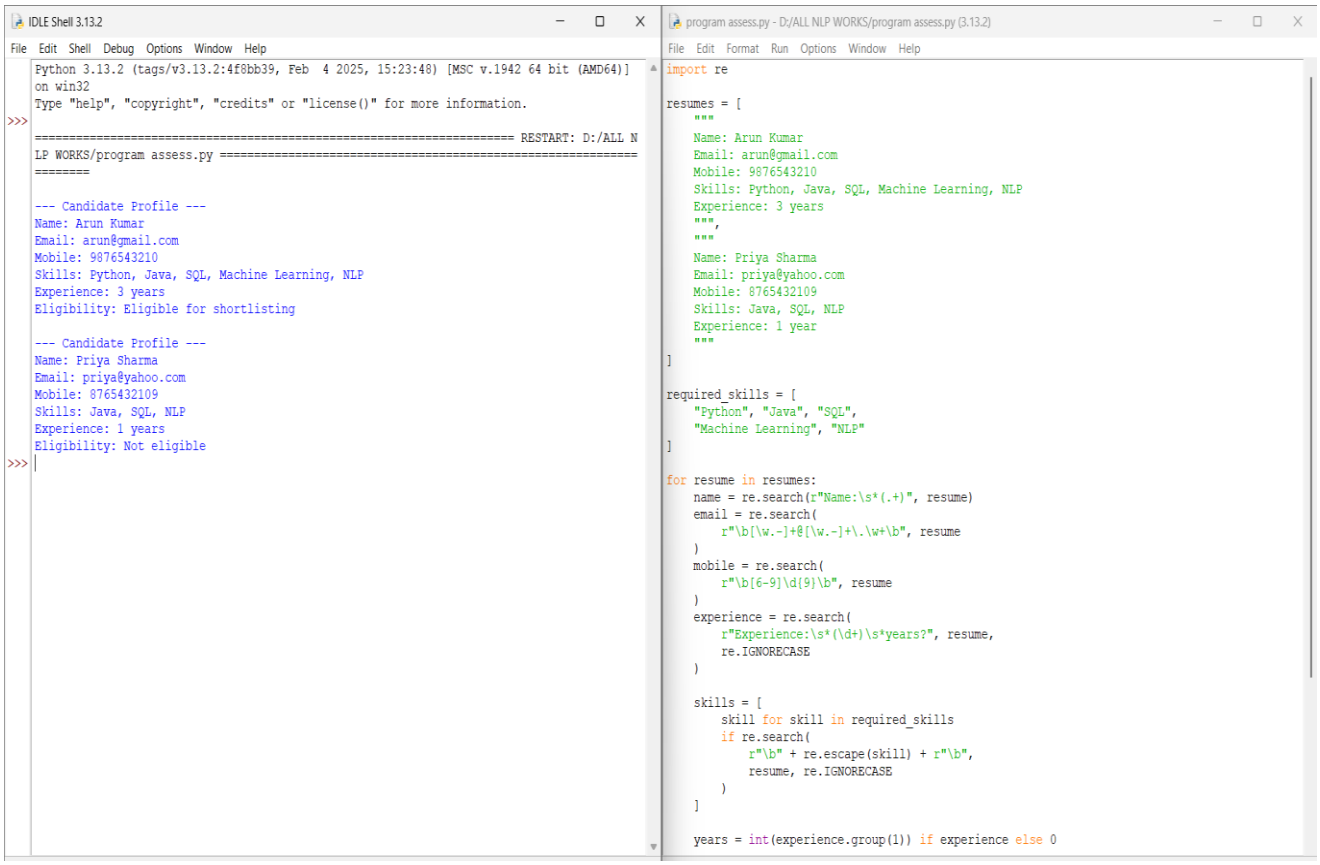
Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

- **Extract the candidate's name.**
- **Identify email addresses.**
- **Extract mobile numbers.**
- **Detect technical skills (Python, Java, SQL, Machine Learning, NLP).**
- **Extract years of experience.**
- **Generate a structured summary of the candidate's profile.**
- **Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).**

Solution :



IDLE Shell 3.13.2

```
Python 3.13.2 (tags/v3.13.2:4f8bb39, Feb  4 2025, 15:23:48) [MSC v.1942 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: D:/ALL NLP WORKS/program assess.py ==========

--- Candidate Profile ---
Name: Arun Kumar
Email: arun@gmail.com
Mobile: 9876543210
Skills: Python, Java, SQL, Machine Learning, NLP
Experience: 3 years
Eligibility: Eligible for shortlisting

--- Candidate Profile ---
Name: Priya Sharma
Email: priya@yahoo.com
Mobile: 8765432109
Skills: Java, SQL, NLP
Experience: 1 years
Eligibility: Not eligible
>>>
```

program assess.py - D:/ALL NLP WORKS/program assess.py (3.13.2)

```
import re

resumes = [
    """
    Name: Arun Kumar
    Email: arun@gmail.com
    Mobile: 9876543210
    Skills: Python, Java, SQL, Machine Learning, NLP
    Experience: 3 years
    """,
    """
    Name: Priya Sharma
    Email: priya@yahoo.com
    Mobile: 8765432109
    Skills: Java, SQL, NLP
    Experience: 1 year
    """
]

required_skills = [
    "Python", "Java", "SQL",
    "Machine Learning", "NLP"
]

for resume in resumes:
    name = re.search(r"Name:\s*(.+)", resume)
    email = re.search(
        r"\b[\w.-]+@[\w.-]+\.\w+\b", resume
    )
    mobile = re.search(
        r"\b[6-9]\d{9}\b", resume
    )
    experience = re.search(
        r"Experience:\s*(\d+)\s*years?", resume,
        re.IGNORECASE
    )

    skills = [
        skill for skill in required_skills
        if re.search(
            r"\b" + re.escape(skill) + r"\b",
            resume, re.IGNORECASE
        )
    ]

    years = int(experience.group(1)) if experience else 0
```

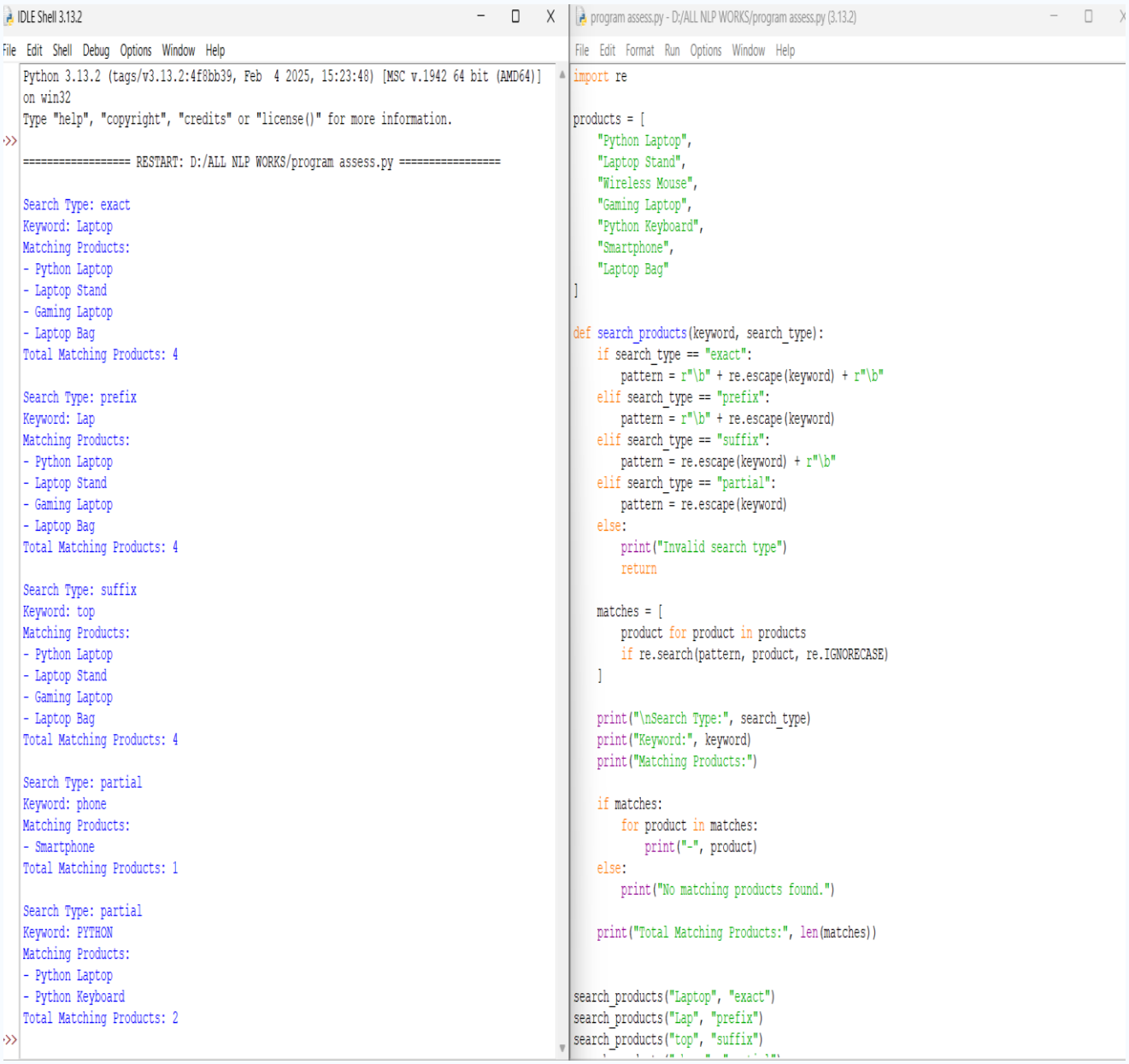
Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

- **Search products using an exact keyword.**
- **Perform prefix-based searches.**
- **Perform suffix-based searches.**
- **Search using partial keywords.**
- **Perform case-insensitive searches.**
- **Display all matching product names.**
- **Generate a report showing the total number of matching products for each search.**

Solution :



IDLE Shell 3.13.2

File Edit Shell Debug Options Window Help

```
Python 3.13.2 (tags/v3.13.2:4f8bb39, Feb  4 2025, 15:23:48) [MSC v.1942 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: D:/ALL NLP WORKS/program assess.py ==========

Search Type: exact
Keyword: Laptop
Matching Products:
- Python Laptop
- Laptop Stand
- Gaming Laptop
- Laptop Bag
Total Matching Products: 4

Search Type: prefix
Keyword: Lap
Matching Products:
- Python Laptop
- Laptop Stand
- Gaming Laptop
- Laptop Bag
Total Matching Products: 4

Search Type: suffix
Keyword: top
Matching Products:
- Python Laptop
- Laptop Stand
- Gaming Laptop
- Laptop Bag
Total Matching Products: 4

Search Type: partial
Keyword: phone
Matching Products:
- Smartphone
Total Matching Products: 1

Search Type: partial
Keyword: PYTHON
Matching Products:
- Python Laptop
- Python Keyboard
Total Matching Products: 2
>>>
```

program assess.py - D:/ALL NLP WORKS/program assess.py (3.13.2)

File Edit Format Run Options Window Help

```
import re

products = [
    "Python Laptop",
    "Laptop Stand",
    "Wireless Mouse",
    "Gaming Laptop",
    "Python Keyboard",
    "Smartphone",
    "Laptop Bag"
]

def search_products(keyword, search_type):
    if search_type == "exact":
        pattern = r"\b" + re.escape(keyword) + r"\b"
    elif search_type == "prefix":
        pattern = r"\b" + re.escape(keyword)
    elif search_type == "suffix":
        pattern = re.escape(keyword) + r"\b"
    elif search_type == "partial":
        pattern = re.escape(keyword)
    else:
        print("Invalid search type")
        return

    matches = [
        product for product in products
        if re.search(pattern, product, re.IGNORECASE)
    ]

    print("\nSearch Type:", search_type)
    print("Keyword:", keyword)
    print("Matching Products:")

    if matches:
        for product in matches:
            print("-", product)
    else:
        print("No matching products found.")

    print("Total Matching Products:", len(matches))


search_products("Laptop", "exact")
search_products("Lap", "prefix")
search_products("top", "suffix")
```

Section 3: Regular Expressions – University Registration System

3. A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

- **Validate the student register number.**
- **Validate the institutional email address.**
- **Validate the course code.**
- **Validate the semester information.**
- **Validate the student's mobile number.**
- **Display appropriate validation messages for each field.**
- **Generate a final registration status report indicating whether the registration is successful.**

Solution :

IDLE Shell 3.13.2

```
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.2 (tags/v3.13.2:4f8bb39, Feb  4 2025, 15:23:48) [MSC v.1942 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>

================ RESTART: D:\ALL NLP WORKS\program assess.py ================
Enter Register Number: 22AD1234
Enter Institutional Email: student@university.edu
Enter Course Code: NLP101
Enter Semester (1-8): 5
Enter Mobile Number: 9344657895

--- Validation Report ---
Register Number: Valid
Institutional Email: Valid
Course Code: Valid
Semester: Valid
Mobile Number: Valid

Registration Status: Successful
>>>
```

program assess.py - D:\ALL NLP WORKS\program assess.py (3.13.2)

```
File  Edit  Format  Run  Options  Window  Help
import re

register_no = input("Enter Register Number: ")
email = input("Enter Institutional Email: ")
course_code = input("Enter Course Code: ")
semester = input("Enter Semester (1-8): ")
mobile = input("Enter Mobile Number: ")

validations = {
    "Register Number": (
        register_no,
        r"^\d{2}[A-Z]{2}\d{4}$"
    ),
    "Institutional Email": (
        email,
        r"^[\w.-]+@university\.edu$"
    ),
    "Course Code": (
        course_code,
        r"^[A-Z]{2,4}\d{3}$"
    ),
    "Semester": (
        semester,
        r"^[1-8]$"
    ),
    "Mobile Number": (
        mobile,
        r"^[6-9]\d{9}$"
    )
}

all_valid = True

print("\n--- Validation Report ---")

for field, (value, pattern) in validations.items():
    if re.fullmatch(pattern, value):
        print(field + ": Valid")
    else:
        print(field + ": Invalid")
        all_valid = False

if all_valid:
    print("\nRegistration Status: Successful")
else:
    print("\nRegistration Status: Failed")
```

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____